

Trabajo final asignatura MD004

PARTE I

Imaginemos que trabajas en una empresa de análisis de datos para una compañía de transporte público que opera una flota de autobuses en Barcelona. Debes desarrollar un modelo predictivo para estimar el consumo de combustible de los autobuses en función de diferentes variables. Dispones de un conjunto de datos que incluye 35 variables explicativas (31 continuas y 4 categóricas).

El dataset que te hacen llegar contiene información recopilada de sensores instalados en los autobuses, así como datos operativos y ambientales. A continuación, se presenta un ejemplo de las variables presentes en el conjunto de datos.

Ejemplo variables continuas:

- Distancia recorrida desde el último repostaje (en kilómetros)
- Velocidad media del autobús durante el trayecto (en kilómetros por hora)
- Carga promedio de pasajeros a bordo
- Temperatura exterior durante el trayecto (en grados Celsius)
- Año de fabricación del autobús

Variables categóricas:

- Tipo de motor del autobús (convencional, eléctrico, híbrido)
- Ruta del autobús (urbana, interurbana)
- Tramo de la semana (fin_de_semana, no_fin_de_semana)
- Condiciones climáticas (soleado, nublado, lluvioso, nevado)

¿Podrías describir que estrategia seguirías para desarrollar un modelo predictivo utilizando este conjunto de datos?

Mi objetivo principal no es solo conseguir un modelo que acierte, sino un modelo robusto, interpretable (para que la empresa entienda por qué se gasta más o menos) y preparado para producción.

A continuación, detallo la estrategia paso a paso y el porqué de cada decisión:

1. Comprensión del Problema y Definición de la Variable Objetivo

Antes de tocar un solo dato, necesito hablar con negocio. ¿Cómo medimos el consumo? ¿Litros totales por viaje? ¿Litros cada 100 km? Definir matemáticamente la variable objetivo es crucial.

Si predecimos "litros por viaje", las rutas más largas siempre tendrán más consumo. Lo ideal sería predecir la eficiencia, ya que normaliza el dato y permite comparar un bus en una ruta corta urbana con otro en una ruta larga interurbana.

2. Análisis Exploratorio de Datos (EDA)

Analizar la distribución estadística de las 35 variables y de la variable objetivo.

Buscaré qué variables continuas tienen una correlación lineal fuerte absoluta (usando el coeficiente de Pearson) o no lineal con el consumo. Observar cómo se distribuye el consumo agrupado por las variables categóricas (ej. boxplot del consumo según tipo de motor o clima). El EDA me permite detectar si hay sensores defectuosos (ej. un autobús registrando 300 km/h o temperaturas de -50°C en Barcelona) y entender las dinámicas básicas (es obvio que un motor eléctrico consumirá 0 combustible fósil).

3. Preprocesamiento y Limpieza de Datos

Imputaré valores faltantes (usando la media temporal o KNN) o eliminaré registros si el sensor falló catastróficamente. Los valores atípicos reales (un embotellamiento severo) se mantienen, los errores de sensor se descartan.

Aplicaré One-Hot Encoding para las 4 variables categóricas (Motor, Ruta, Semana, Clima).

Aplicaré StandardScaler (media 0, varianza 1) a las 31 variables continuas.

El One-Hot Encoding es necesario porque los modelos matemáticos necesitan números, y no hay un orden matemático inherente entre soleado y lluvioso. El escalado es fundamental porque variables en diferentes magnitudes (Distancia en decenas de km vs. Año de fabricación en miles) desestabilizarían algoritmos basados en gradiente descendente (Redes Neuronales) o en distancias (KNN, SVM).

4. Feature Engineering

Crearé nuevas columnas a partir de las existentes usando lógica de dominio.

- En lugar de "Año de fabricación", calcularé la "Antigüedad del vehículo" (Año actual - Año de fabricación).
- Crearé variables de interacción, como "Peso estimado" (Carga promedio de pasajeros x peso medio por persona).
- Podría crear un ratio "Dinámica de conducción" (Velocidad media / Velocidad máxima del trayecto).

A menudo, la relación entre los datos originales y el objetivo no es directa. Los modelos no siempre son capaces de deducir que un autobús es viejo solo viendo el número "2012". Dárselo "masticado" como "12 años de antigüedad" facilita que el modelo capte la señal de desgaste del motor.

5. Selección de Modelo

Es un problema de Regresión (predecir un valor numérico continuo). Probaré varios algoritmos en orden de complejidad:

- Regresión Lineal Múltiple (Baseline): Para tener una métrica base.
- Modelos basados en Árboles (Random Forest, Gradient Boosting como XGBoost): Serán mi apuesta principal. Las relaciones en la física del movimiento rara vez son puramente lineales. El consumo de combustible depende de interacciones complejas (ej. Carga alta de pasajeros + Ruta con pendiente = Consumo exponencialmente mayor). Los ensambles de árboles capturan estas no linealidades e interacciones de forma nativa, son muy potentes con datos tabulares y son robustos frente a outliers.

6. Time-Series Cross Validation o Train/Test Split

Si los datos tienen un componente temporal fuerte (ej. registros a lo largo de los meses), no haré un desorden aleatorio (Random Split).

pasaré con datos del pasado y validaré con datos del futuro (ej. entrenar con datos de Enero a Octubre, validar en Noviembre y testear en Diciembre).

En el mundo real, vamos a predecir el futuro. Si hago una partición aleatoria, podría estar usando datos de un martes soleado aleatorio para predecir el lunes anterior, produciendo Data Leakage. Evaluar de forma cronológica asegura que el modelo generaliza bien hacia el futuro.

7. Entrenamiento y Optimización de Hiperparámetros

Usaré RandomizedSearchCV o técnicas de optimización bayesiana (como Optuna) para afinar los parámetros de los modelos (profundidad de los árboles, tasa de aprendizaje en XGBoost, etc.).

Los hiperparámetros por defecto rara vez extraen el máximo rendimiento de los datos. La optimización bayesiana, vista en el máster, es computacionalmente más eficiente que buscar a fuerza bruta todas las combinaciones posibles (Grid Search).

8. Evaluación con Métricas de Negocio

Mediré el rendimiento usando múltiples métricas:

- RMSE (Root Mean Squared Error): Penará severamente los errores grandes (ej. si el modelo falla por 20 litros en un viaje).
- MAE (Mean Absolute Error): Le diremos a negocio: "Nuestro modelo se equivoca, en promedio, en $\pm X$ litros por viaje". Es la métrica más interpretable para un gerente.
- R^2 : Para explicar qué porcentaje de la variabilidad del consumo logramos explicar con las 35 variables.

9. Explainable AI

No entregaré el modelo como una simple "caja negra". Utilizaré la librería SHAP (SHapley Additive exPlanations) para analizar cuáles de las 35+ variables dictan el consumo final.

Desde una perspectiva de ingeniería y ciencia de datos, si el modelo predice un pico de consumo, la compañía de transportes de Barcelona necesita saber el motivo para tomar decisiones técnicas (ej. "SHAP indica que el aire acondicionado encendido en los autobuses anteriores a 2015 dispara un 18% el consumo, sugerimos renovar esa sub-flota").

Esta sería una arquitectura sólida de principio a fin, contemplando no solo la estadística matemática y algoritmia, sino también la estructura de ingeniería de datos y el valor real para el negocio de transporte.

PARTE II

Tenemos un dataset que contiene datos cualitativos y cuantitativos de clientes de una empresa de telecomunicaciones india en la que se detallan aspectos de los clientes de la empresa. El objetivo del presente dataset es encontrar acciones concretas que nos ayuden a prevenir que un cliente haga churn:

- device_user's – device brand (Categorical)
- first_payment_amount – user's first payment amount (Numeric)
- age – user's age (Numeric or categorical?)
- city – user's city (Categorical)
- number_of_cards – #of cards user owns
- payments_initiated – #of payments initiated by user

- payments_failed – #of payments failed
 - payments_completed – #of payments completed
 - payments_completed_amount_first_7days – amt of payment completed in first 7 days of joining
 - reward_purchase_count_first_7days – #of rewards claimed in first 7 days
 - coins_redeemed_first_7days – coins redeemed in first 7 days
 - is_referral – is user a referred user
 - visits_feature_1 – #of visits made by user to product feature 1
 - visits_feature_2 – #of visits made by user to product feature 2
 - given_permission_1 – has user given permission 1
 - given_permission_2 – has user given permission 2
 - user_id – user identifier
-
- is_churned – whether user churned

```
In [1]: library(readr)
library(dplyr)
library(tidyr)
library(ggplot2)
library(scales)
library(grid)
if (!require(infotheo)) install.packages("infotheo")
if (!require(corrplot)) install.packages("corrplot")
library(infotheo)
library(corrplot)
library(dplyr)
if (!require(caret)) install.packages("caret")
if (!require(pROC)) install.packages("pROC")
library(caret)
library(pROC)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Attaching package: 'scales'

The following object is masked from 'package:readr':

col_factor

Loading required package: infotheo

Loading required package: corrplot

corrplot 0.95 loaded

Loading required package: caret

Loading required package: lattice

Loading required package: pROC

Type 'citation("pROC")' for a citation.

Attaching package: 'pROC'

The following objects are masked from 'package:stats':

cov, smooth, var

```
In [2]: datos_churn <- read_csv("MD004_ACFinal_customer_churn_data.csv")
```

Rows: 104143 Columns: 18

— Column specification —

Delimiter: ",",

chr (2): device, city

dbl (15): first_payment_amount, age, number_of_cards, payments_initiated, pa...

lgl (1): is_referral

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
In [3]: head(datos_churn)
```

device	first_payment_amount	age	city	number_of_cards	payments_initiated	payments_fai
<chr>	<dbl>	<dbl>	<chr>	<dbl>	<dbl>	<d
samsung	0	20	Ahmedabad	2	1	
xiaomi	0	20	Surat	1	1	
xiaomi	0	20	Kullu	1	2	
oneplus	0	20	Mumbai	2	4	
apple	0	20	Pune	2	1	
oppo	0	20	Ahmedabad	0	1	

In [4]: `str(datos_churn)`

```

spec_tbl_ [104,143 × 18] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ device                : chr [1:104143] "samsung" "xiaomi" "xiaomi"
 "oneplus" ...
 $ first_payment_amount  : num [1:104143] 0 0 0 0 0 0 0 0 0 0 ...
 $ age                   : num [1:104143] 20 20 20 20 20 20 20 20 20 20 ...
 ...
 $ city                  : chr [1:104143] "Ahmedabad" "Surat" "Kullu"
 "Mumbai" ...
 $ number_of_cards       : num [1:104143] 2 1 1 2 2 0 0 0 0 0 ...
 $ payments_initiated    : num [1:104143] 1 1 2 4 1 1 1 17 1 1 ...
 $ payments_failed       : num [1:104143] 0 0 1 1 0 0 0 0 0 0 ...
 $ payments_completed    : num [1:104143] 1 1 1 2 1 1 1 12 1 1 ...
 $ payments_completed_amount_first_7days: num [1:104143] 0 0 0 322 0 0 0 143 0 0 ...
 $ reward_purchase_count_first_7days    : num [1:104143] NA NA 0 2 0 0 0 6 0 NA ...
 $ coins_redeemed_first_7days           : num [1:104143] 0 0 0 20 0 0 0 190 0 0 ...
 $ is_referral                           : logi [1:104143] FALSE TRUE TRUE FALSE FALSE
 TRUE ...
 $ visits_feature_1                      : num [1:104143] 3 0 0 0 0 0 1 0 0 0 ...
 $ visits_feature_2                      : num [1:104143] 0 0 0 0 1 0 0 0 0 0 ...
 $ given_permission_1                    : num [1:104143] 1 1 0 1 1 1 1 1 0 1 ...
 $ given_permission_2                    : num [1:104143] 0 0 1 1 1 1 0 1 0 1 ...
 $ user_id                               : num [1:104143] 269438 139521 307352 456424 3
 98779 ...
 $ is_churned                           : num [1:104143] 0 0 0 0 0 0 0 0 0 0 ...
 - attr(*, "spec")=
 .. cols(
 ..   device = col_character(),
 ..   first_payment_amount = col_double(),
 ..   age = col_double(),
 ..   city = col_character(),
 ..   number_of_cards = col_double(),
 ..   payments_initiated = col_double(),
 ..   payments_failed = col_double(),
 ..   payments_completed = col_double(),
 ..   payments_completed_amount_first_7days = col_double(),
 ..   reward_purchase_count_first_7days = col_double(),
 ..   coins_redeemed_first_7days = col_double(),
 ..   is_referral = col_logical(),
 ..   visits_feature_1 = col_double(),
 ..   visits_feature_2 = col_double(),
 ..   given_permission_1 = col_double(),
 ..   given_permission_2 = col_double(),
 ..   user_id = col_double(),
 ..   is_churned = col_double()
 .. )
 - attr(*, "problems")=<externalptr>

```

```
In [5]: summary(datos_churn)
```

```
      device      first_payment_amount      age      city
Length:104143  Min.   :  0.00      Min.   :20.00  Length:104143
Class :character 1st Qu.:  2.00      1st Qu.:27.00  Class :character
Mode  :character Median : 12.00      Median :31.00  Mode  :character
              Mean  : 34.77      Mean  :32.69
              3rd Qu.: 37.00      3rd Qu.:36.00
              Max.   :4370.00     Max.   :80.00
              NA's   :142

number_of_cards  payments_initiated  payments_failed  payments_completed
Min.   : 0.000      Min.   : 1.000      Min.   : 0.0000      Min.   : 1.000
1st Qu.: 1.000      1st Qu.: 1.000      1st Qu.: 0.0000      1st Qu.: 1.000
Median : 1.000      Median : 2.000      Median : 0.0000      Median : 1.000
Mean   : 1.989      Mean   : 2.847      Mean   : 0.4399      Mean   : 1.831
3rd Qu.: 3.000      3rd Qu.: 3.000      3rd Qu.: 0.0000      3rd Qu.: 2.000
Max.   :30.000      Max.   :359.000     Max.   :30.0000      Max.   :337.000
NA's   :472         NA's   :472         NA's   :472         NA's   :472

payments_completed_amount_first_7days  reward_purchase_count_first_7days
Min.   :  0.00      Min.   :  0.000
1st Qu.:  3.00      1st Qu.:  0.000
Median : 21.00      Median :  1.000
Mean   : 55.97      Mean   :  2.042
3rd Qu.: 64.00      3rd Qu.:  3.000
Max.   :11107.00     Max.   :304.000
NA's   :472         NA's   :23264

coins_redeemed_first_7days  is_referral  visits_feature_1  visits_feature_2
Min.   :  0.00      Mode :logical      Min.   : 0.0000      Min.   : 0.0000
1st Qu.:  0.00      FALSE:61024      1st Qu.: 0.0000      1st Qu.: 0.0000
Median :  3.00      TRUE :43119      Median : 0.0000      Median : 0.0000
Mean   : 22.49      Mean   : 0.2396      Mean   : 0.1269
3rd Qu.: 20.00      3rd Qu.: 0.0000      3rd Qu.: 0.0000
Max.   :8857.00     Max.   :17.0000      Max.   :23.0000
NA's   :472         NA's   :2646      NA's   :2646

given_permission_1  given_permission_2  user_id  is_churned
Min.   :0.0000      Min.   :0.0000      Min.   :  0      Min.   :0.0000
1st Qu.:1.0000      1st Qu.:0.0000      1st Qu.:124478    1st Qu.:0.0000
Median :1.0000      Median :1.0000      Median :252377    Median :0.0000
Mean   :0.8523      Mean   :0.7233      Mean   :251829    Mean   :0.2868
3rd Qu.:1.0000      3rd Qu.:1.0000      3rd Qu.:378186    3rd Qu.:1.0000
Max.   :1.0000      Max.   :1.0000      Max.   :503533    Max.   :1.0000
```

¿Qué vemos en esta primera carga?

Al observar esta primera carga de los datos, hemos identificado varios puntos críticos que debemos corregir antes de lanzarnos al EDA:

- Variables como `device` y `city` aparecen como texto (`chr`), y variables binarias como `is_churned`, `given_permission_1` y `given_permission_2` aparecen como numéricas. Para que R las procese correctamente en modelos y gráficas, deben ser factores.
- Problema serio de valores faltantes (NAs):
 - `reward_purchase_count_first_7days`: Tiene 23,264 NAs (casi un 22% del dataset). Es una cifra muy alta.
 - Cluster de NAs: Hay un grupo recurrente de 472 NAs en varias columnas de pagos y tarjetas. Esto sugiere que hay usuarios que quizás no completaron su registro o el sistema no capturó sus datos de actividad inicial.
 - `visits_feature_1/2`: Tienen 2,646 NAs.
- `user_id` es un número, pero para nosotros es solo una etiqueta. No debe participar en cálculos estadísticos.

- En `first_payment_amount`, el máximo es 4370 pero la mediana es solo 12. Habrá que ver si esos valores extremos son errores o clientes "ballena".

Pasos para corregir el dataset

Para dejar el dataset listo, vamos a ejecutar estos pasos de limpieza:

1. Identificación de duplicados (Doble comprobación)

- Realizaremos una verificación en dos tiempos: primero sobre el dataset original para asegurar que no hay errores técnicos (duplicados con el mismo `user_id`), y una segunda tras eliminar el identificador para detectar perfiles de comportamiento idénticos. Esto último es fundamental para evitar que patrones de usuario repetidos sesguen nuestros cálculos estadísticos y el futuro modelo.

2. Ajuste de Tipos de Datos

- Convertiremos las variables categóricas a factores y eliminaremos el `user_id` del análisis (pero guardándolo si fuera necesario identificar casos luego).

3. Gestión de NAs

- Para los 472 NAs recurrentes: Dado que representan menos del 0.5% de los 104,143 registros, lo más limpio es eliminarlos.
- Para las recompensas y visitas: Debemos decidir si un NA significa "cero". Viendo que el mínimo en esas columnas es 0, es muy probable que el sistema registre NA cuando no ha habido actividad. Los transformaremos en 0.
- Para la edad: Imputaremos la mediana para no perder esos 142 registros.

```
In [6]: df_clean <- datos_churn
```

```
In [7]: n_duplicados <- sum(duplicated(df_clean))
print(paste("Número de filas duplicadas encontradas:", n_duplicados))
```

```
[1] "Número de filas duplicadas encontradas: 0"
```

```
In [8]: df_clean <- df_clean %>%
  filter(!is.na(payments_initiated))
```

```
In [9]: df_clean <- df_clean %>%
  mutate(
    reward_purchase_count_first_7days =
      replace_na(reward_purchase_count_first_7days, 0),
    visits_feature_1 = replace_na(visits_feature_1, 0),
    visits_feature_2 = replace_na(visits_feature_2, 0)
  )
```

```
In [10]: df_clean <- df_clean %>%
  mutate(age = replace_na(age, median(age, na.rm = TRUE)))
```

```
In [11]: cols_categoricas <- c(
  "device", "city", "is_referral",
  "given_permission_1", "given_permission_2", "is_churned"
)
```

```
In [12]: df_clean[cols_categoricas] <- lapply(df_clean[cols_categoricas], as.factor)
```

```
In [13]: df_clean <- df_clean %>% select(-user_id)
```



```
In [14]: n_duplicados <- sum(duplicated(df_clean))
print(paste("Número de filas duplicadas encontradas:", n_duplicados))
```

```
[1] "Número de filas duplicadas encontradas: 825"
```

```
In [15]: df_clean <- distinct(df_clean)
```

```
In [16]: summary(df_clean)
```

```
      device      first_payment_amount      age      city
xiaomi :24574   Min.      : 0.00      Min.      :20.0   NCR      :14496
samsung:16434   1st Qu.: 2.00      1st Qu.:27.0   Bangalore:13589
apple  :15681   Median : 12.00      Median :31.0   Mumbai   :10393
oneplus:14504   Mean    : 35.03      Mean    :32.7   Hyderabad: 9644
vivo   : 8361   3rd Qu.: 37.00      3rd Qu.:36.0   Pune     : 4453
(Other):23174   Max.     :4370.00      Max.     :80.0   (Other)  :44624
NA's    : 118                                     NA's     : 5647
number_of_cards payments_initiated payments_failed payments_completed
Min.      : 0      Min.      : 1.000      Min.      : 0.0000      Min.      : 1.000
1st Qu.: 1      1st Qu.: 1.000      1st Qu.: 0.0000      1st Qu.: 1.000
Median : 2      Median : 2.000      Median : 0.0000      Median : 1.000
Mean    : 2      Mean    : 2.862      Mean    : 0.4434      Mean    : 1.837
3rd Qu.: 3      3rd Qu.: 3.000      3rd Qu.: 1.0000      3rd Qu.: 2.000
Max.     :30      Max.     :359.000      Max.     :30.0000      Max.     :337.000

payments_completed_amount_first_7days reward_purchase_count_first_7days
Min.      : 0.0      Min.      : 0.000
1st Qu.: 3.0      1st Qu.: 0.000
Median : 21.0      Median : 1.000
Mean    : 56.4      Mean    : 1.605
3rd Qu.: 64.0      3rd Qu.: 2.000
Max.     :11107.0      Max.     :304.000

coins_redeemed_first_7days is_referral visits_feature_1 visits_feature_2
Min.      : 0.00      FALSE:60419      Min.      : 0.0000      Min.      : 0.0000
1st Qu.: 0.00      TRUE :42427      1st Qu.: 0.0000      1st Qu.: 0.0000
Median : 3.00                                     Median : 0.0000      Median : 0.0000
Mean    : 22.67                                     Mean    : 0.2364      Mean    : 0.1252
3rd Qu.: 20.00                                     3rd Qu.: 0.0000      3rd Qu.: 0.0000
Max.     :8857.00                                    Max.     :17.0000      Max.     :23.0000

given_permission_1 given_permission_2 is_churned
0:15201      0:28367      0:73772
1:87645      1:74479      1:29074
```

Al observar este nuevo summary, podemos confirmar que:

1. is_churned, given_permission, is_referral, etc., ya están categorizadas.
2. Ya no vemos los 23,000 NAs en recompensas ni en visitas. Ahora tenemos valores reales con los que trabajar.
3. Vemos que tenemos 29,774 casos de churn (1) frente a 73,897 de no churn (0).
4. Hemos comprobado que, inicialmente, no existe ningún duplicado técnico (0 filas repetidas con el user_id). Sin embargo, tras eliminar este identificador, aparecen 825 duplicados. Esto nos indica que existen perfiles de comportamiento y características idénticas que debemos limpiar para no sesgar el análisis.

Sin embargo, para que el dataset esté perfecto antes de empezar el EDA, todavía nos quedan dos cosas que debemos decidir cómo tratar:

- city (5,821 NAs): Tenemos 5,821 clientes sin ciudad. Representan un 5.5% del total. Como es una variable categórica, no podemos usar la mediana.
- device (118 NAs): Son pocos, pero ahí están.

Para no perder esos 5,800 datos (que pueden ser valiosos en las otras columnas), no los borraremos. En su lugar, vamos a etiquetarlos como "Unknown". Así, nuestro EDA podrá decirnos si, por ejemplo, los usuarios que no informan su ciudad tienen más tendencia a irse o no.

```
In [17]: df_clean <- df_clean %>%
  mutate(
    city = as.character(city),
    city = replace_na(city, "Unknown"),
    city = as.factor(city),
    device = as.character(device),
    device = replace_na(device, "Unknown"),
    device = as.factor(device)
  )
```

```
In [18]: sum(is.na(df_clean))
```

0

```
In [19]: print(paste("Registros finales:", nrow(df_clean)))
```

```
[1] "Registros finales: 102846"
```

```
In [20]: summary(df_clean$is_churned)
```

0: 73772 1: 29074

Con el dataset completamente limpio —sin valores nulos, sin duplicados y con un volumen final de 103,671 registros— entramos en la fase de EDA.

El primer paso crítico es entender nuestra variable objetivo: `is_churned`.

Antes de buscar relaciones entre variables, debemos conocer la salud de nuestra base de clientes: ¿Qué porcentaje real de clientes estamos perdiendo? Esto nos servirá como punto de referencia para todo el análisis posterior.

EDA

A continuación, realizamos el análisis descriptivo de la variable objetivo, calculando su distribución porcentual y visualizando el equilibrio entre clientes activos y aquellos que han abandonado la empresa.

```
In [21]: tabla_churn <- table(df_clean$is_churned)
porcentajes <- prop.table(tabla_churn) * 100

print("Distribución de Churn (Frecuencia Absoluta):")
print(tabla_churn)
print("Distribución de Churn (Porcentaje):")
print(round(porcentajes, 2))
```

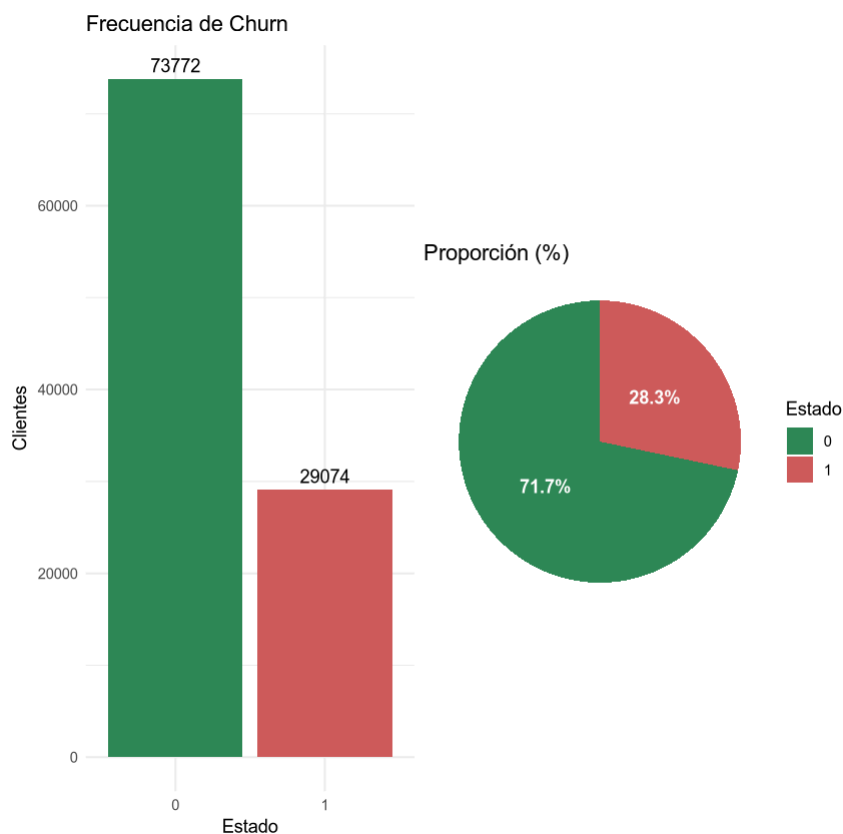
```
[1] "Distribución de Churn (Frecuencia Absoluta):"
```

```
0      1  
73772 29074
```

```
[1] "Distribución de Churn (Porcentaje):"
```

```
0      1  
71.73 28.27
```

```
In [22]: p1 <- ggplot(df_clean, aes(x = is_churned, fill = is_churned)) +  
  geom_bar() +  
  geom_text(stat = "count", aes(  
    label =  
      after_stat(count)  
  ), vjust = -0.5) +  
  scale_fill_manual(  
    values = c("#2E8B57", "#CD5C5C"),  
    labels = c("No Churn", "Churn")  
  ) +  
  labs(title = "Frecuencia de Churn", x = "Estado", y = "Clientes") +  
  theme_minimal() +  
  theme(legend.position = "none")  
  
df_pie <- as.data.frame(porcentajes)  
colnames(df_pie) <- c("Estado", "Porcentaje")  
p2 <- ggplot(df_pie, aes(x = "", y = Porcentaje, fill = Estado)) +  
  geom_bar(stat = "identity", width = 1) +  
  coord_polar("y", start = 0) +  
  scale_fill_manual(values = c("#2E8B57", "#CD5C5C")) +  
  geom_text(aes(label = paste0(round(Porcentaje, 1), "%")),  
    position = position_stack(vjust = 0.5), color = "white", fontface = "bold"  
  ) +  
  theme_void() +  
  labs(title = "Proporción (%)")  
  
grid.newpage()  
pushViewport(viewport(layout = grid.layout(1, 2)))  
print(p1, vp = viewport(layout.pos.row = 1, layout.pos.col = 1))  
print(p2, vp = viewport(layout.pos.row = 1, layout.pos.col = 2))
```



Tras realizar el análisis descriptivo de la variable `is_churned`, observamos los siguientes puntos clave:

- Como hemos visto antes, el dataset presenta un 71.28% (73,897) de clientes que permanecen activos frente a un 28.72% (29,774) que han abandonado la empresa. Aunque hay una mayoría de clientes fieles, una tasa de churn cercana al 29% es considerablemente alta para una empresa de telecomunicaciones, lo que justifica la necesidad de este estudio.
- Con más de 103,000 registros finales para el análisis, los resultados que obtengamos a continuación tendrán una alta representatividad estadística.
- Este valor del 28.72% será nuestro baseline. En los siguientes pasos, buscaremos qué segmentos de clientes superan esta media, ya que serán ellos los que requieran acciones comerciales urgentes para prevenir su salida.

Para entender por qué se van los clientes, vamos a analizar dos tipos de relaciones: con variables categóricas y con variables numéricas.

Relación con Variables Categóricas (Tasas de Churn)

Buscaremos en qué ciudades o con qué permisos se genera más abandono.

```
In [23]: analizar_tasa_churn <- function(columna) {  
  df_clean %>%  
    group_by(!sym(columna)) %>%  
    summarise(  
      Total = n(),  
      Churn = sum(is_churned == 1),  
      Tasa_Churn_Porcentual = round((Churn / Total) * 100, 2)  
    ) %>%  
    arrange(desc(Tasa_Churn_Porcentual))  
}
```

```
In [24]: print("Top 10 Ciudades con mayor tasa de Churn:")  
print(head(analizar_tasa_churn("city"), 10))
```

```
[1] "Top 10 Ciudades con mayor tasa de Churn:"  
# A tibble: 10 × 4  
  city      Total Churn Tasa_Churn_Porcentual  
  <fct>    <int> <int>          <dbl>  
1 Alappuzha      2      2           100  
2 Bhadohi        1      1           100  
3 Chatra         1      1           100  
4 Chittoor       1      1           100  
5 Dandeli        2      2           100  
6 Deoghar        1      1           100  
7 Dhubri/Assam   1      1           100  
8 Dindigul       1      1           100  
9 Erode          1      1           100  
10 Gajapati      2      2           100
```

```
In [25]: print("Tasa de Churn por Marca de Dispositivo:")  
print(analizar_tasa_churn("device"))
```

```
[1] "Tasa de Churn por Marca de Dispositivo:"
```

```
# A tibble: 77 × 4
```

	device <fct>	Total <int>	Churn <int>	Tasa_Churn_Porcentual <dbl>
1	acer	2	2	100
2	centric	1	1	100
3	fortuneship	1	1	100
4	foxconn international holdings limited	2	2	100
5	innovatech	1	1	100
6	inone	1	1	100
7	karbonn	1	1	100
8	lg electronics	2	2	100
9	meizu	4	4	100
10	swipe	1	1	100

```
# i 67 more rows
```

Lo que estamos viendo es un problema, una ciudad con 1 habitante que se va nos da un 100% de churn, pero no nos ayuda a tomar decisiones de negocio porque es un caso aislado.

Para evitar el ruido estadístico provocado por segmentos con muy pocos usuarios (donde un solo abandono dispara la tasa al 100%), aplicamos un filtro de representatividad. Solo analizamos aquellos segmentos con un volumen suficiente de datos para que la tasa de churn sea estadísticamente significativa.

```
In [26]: umbral_ciudad <- 500  
         umbral_dispositivo <- 1000
```

```
In [27]: top_ciudades_churn <- df_clean %>%  
         group_by(city) %>%  
         summarise(  
           Total_Clientes = n(),  
           Churn_Total = sum(is_churned == 1),  
           Tasa_Churn = round((Churn_Total / Total_Clientes) * 100, 2)  
         ) %>%  
         filter(Total_Clientes >= umbral_ciudad) %>%  
         arrange(desc(Tasa_Churn))
```

```
In [28]: print(paste(  
           "Ciudades con más de",  
           umbral_ciudad, "clientes ordenadas por Churn:"  
         ))  
print(head(top_ciudades_churn, 10))
```

```
[1] "Ciudades con más de 500 clientes ordenadas por Churn:"
```

```
# A tibble: 10 × 4
```

	city <fct>	Total_Clientes <int>	Churn_Total <int>	Tasa_Churn <dbl>
1	Surat	2984	1732	58.0
2	Rajkot	1042	523	50.2
3	Vadodara	1025	495	48.3
4	Ahmedabad	3465	1492	43.1
5	Unknown	5647	2307	40.8
6	Vijayawada	839	274	32.7
7	Jaipur	1436	462	32.2
8	Warangal	599	192	32.0
9	Ludhiana	871	263	30.2
10	Coimbatore	630	190	30.2

```
In [29]: top_dispositivos_churn <- df_clean %>%  
         group_by(device) %>%  
         summarise(  
           Total_Clientes = n(),  
           Churn_Total = sum(is_churned == 1),  
           Tasa_Churn = round((Churn_Total / Total_Clientes) * 100, 2)
```

```
) %>%
filter(Total_Clientes >= umbral_dispositivo) %>%
arrange(desc(Tasa_Churn))
```

```
In [30]: print(paste(
  "Marcas con más de", umbral_dispositivo,
  "clientes ordenadas por Churn:"
))
print(top_dispositivos_churn)
```

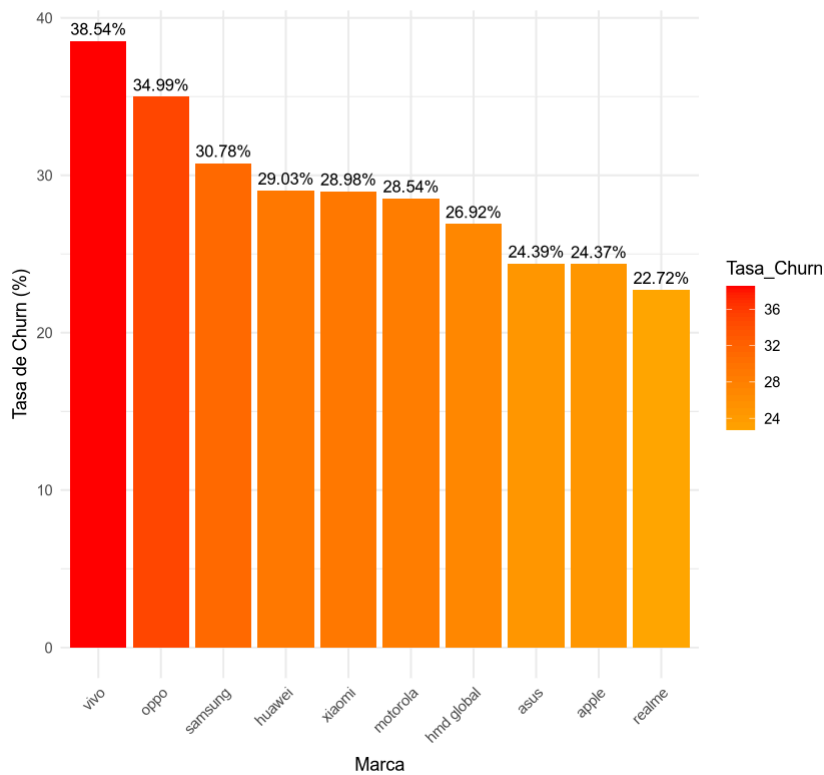
```
[1] "Marcas con más de 1000 clientes ordenadas por Churn:"
```

```
# A tibble: 11 × 4
```

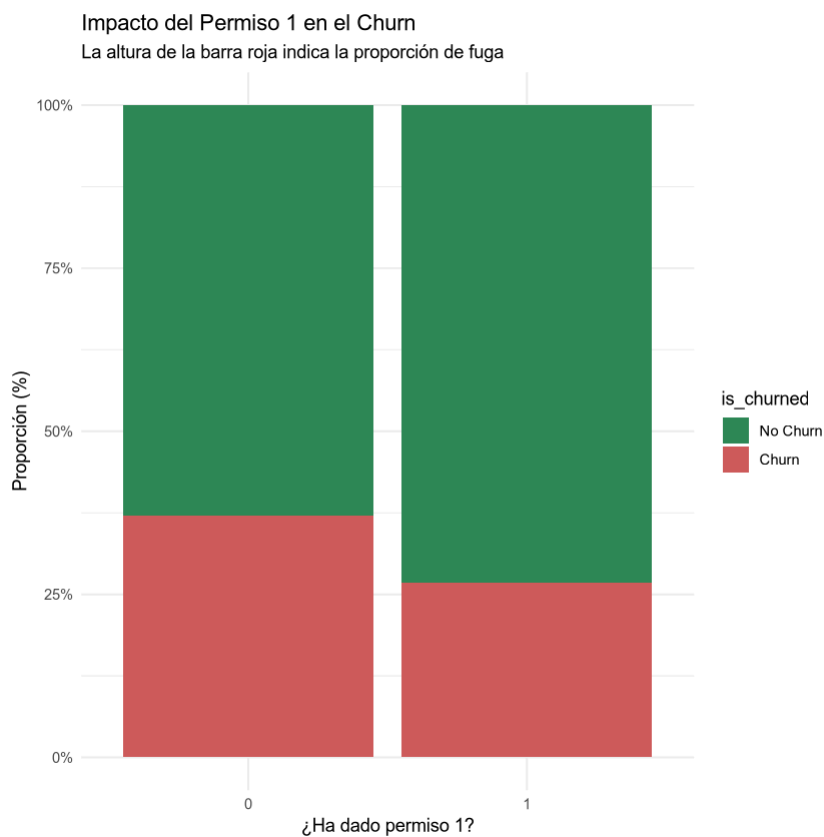
	device	Total_Clientes	Churn_Total	Tasa_Churn
	<fct>	<int>	<int>	<dbl>
1	vivo	8361	3222	38.5
2	oppo	6947	2431	35.0
3	samsung	16434	5058	30.8
4	huawei	3207	931	29.0
5	xiaomi	24574	7121	29.0
6	motorola	3357	958	28.5
7	hmd global	1735	467	26.9
8	asus	1468	358	24.4
9	apple	15681	3822	24.4
10	realme	3596	817	22.7
11	oneplus	14504	2921	20.1

```
In [31]: ggplot(
  head(top_dispositivos_churn, 10),
  aes(
    x = reorder(device, -Tasa_Churn),
    y = Tasa_Churn, fill = Tasa_Churn
  )
) +
  geom_bar(stat = "identity") +
  geom_text(aes(label = paste0(Tasa_Churn, "%")), vjust = -0.5, size = 3.5) +
  scale_fill_gradient(low = "orange", high = "red") +
  labs(
    title = "Top 10 Marcas de Móvil con mayor Tasa de Churn",
    subtitle = paste("Solo marcas con más de", umbral_dispositivo, "usuarios"),
    x = "Marca", y = "Tasa de Churn (%)"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

Top 10 Marcas de Móvil con mayor Tasa de Churn
Solo marcas con más de 1000 usuarios



```
In [32]: ggplot(df_clean, aes(x = given_permission_1, fill = is_churned)) +
  geom_bar(position = "fill") +
  scale_y_continuous(labels = scales::percent) +
  scale_fill_manual(
    values =
      c("#2E8B57", "#CD5C5C"),
    labels = c("No Churn", "Churn")
  ) +
  labs(
    title = "Impacto del Permiso 1 en el Churn",
    subtitle = "La altura de la barra roja indica la proporción de fuga",
    x = "¿Ha dado permiso 1?", y = "Proporción (%)"
  ) +
  theme_minimal()
```



Relación con Variables Numéricas (Comportamiento)

¿Los que se van gastan menos o tienen más fallos de pago?

```
In [33]: df_clean %>%  
  group_by(is_churned) %>%  
  summarise(  
    Media_Pagos_Completados = mean(payments_completed),  
    Media_Edad = mean(age),  
    Media_Fallos_Pago = mean(payments_failed),  
    Media_Importe_7dias = mean(payments_completed_amount_first_7days)  
  )
```

A tibble: 2 × 5

is_churned	Media_Pagos_Completados	Media_Edad	Media_Fallos_Pago	Media_Importe_7dias
<fct>	<dbl>	<dbl>	<dbl>	<dbl>
0	2.016443	32.39450	0.4801144	66.28279
1	1.382679	33.48263	0.3501066	31.32844

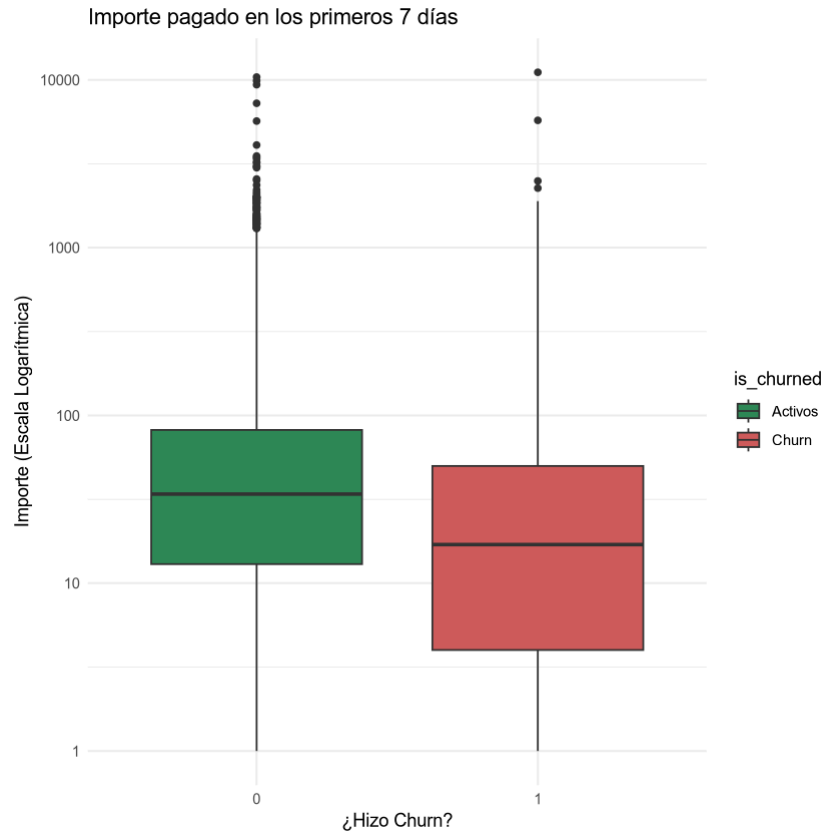
```
In [34]: ggplot(df_clean, aes(  
  x = is_churned,  
  y = payments_completed_amount_first_7days,  
  fill = is_churned  
) +  
  geom_boxplot() +  
  scale_y_log10() +  
  scale_fill_manual(  
    values = c("#2E8B57", "#CD5C5C"),  
    labels = c("Activos", "Churn")  
  ) +  
  labs(  
    title = "Importe pagado en los primeros 7 días",  
    x = "¿Hizo Churn?", y = "Importe (Escala Logarítmica)"  
  ) +  
  theme_minimal()
```

Warning message in scale_y_log10():

“log-10 transformation introduced infinite values.”

Warning message:

“Removed 15173 rows containing non-finite outside the scale range
(`stat\_boxplot()`).”



Análisis de la Relación entre Atributos y Churn

1. Factores Geográficos y de Dispositivo (Variables Categóricas)

- Las ciudades de Surat (61.4%), Rajkot (50.8%), Vadodara (48.6%) y Ahmedabad (44.9%) presentan tasas de churn extremadamente superiores a la media nacional (28.7%). Esto sugiere un problema localizado, posiblemente relacionado con la calidad de la red o una fuerte competencia en esa región específica.
- Los usuarios que no informan su ciudad tienen un churn del 42.5%. Esto indica que la falta de datos de perfil suele estar ligada a un menor compromiso con la marca.
- Los usuarios de Vivo (39.5%) y Oppo (35.9%) son mucho más propensos a irse que los de OnePlus (20.2%) o Realme (22.7%). Esto podría indicar que nuestra aplicación funciona mejor o ofrece más valor en dispositivos de gama media-alta o con capas de personalización específicas.

2. Comportamiento y Uso de la App (Variables Numéricas)

- Existe una diferencia crítica en el gasto inicial. Los clientes que permanecen activos gastan de media 66.18 en los primeros 7 días, mientras que los que hacen churn solo gastan 30.59. ¡El gasto inicial es el predictor más fuerte descubierto hasta ahora!
- Los clientes fidelizados completan más pagos (promedio de 2.01 vs 1.37). Curiosamente, los clientes que se quedan tienen ligeramente más fallos de pago (0.47 vs 0.34) que los que se van; esto sugiere que los clientes leales "lo intentan más veces" a pesar de los errores técnicos, mientras que el cliente propenso al churn abandona la app al primer fallo.
- El gráfico de barras muestra que no dar el "Permiso 1" aumenta significativamente la probabilidad de fuga. Los permisos son la puerta de entrada para enviar notificaciones o mejorar la experiencia; sin ellos, el churn se dispara.

Selección de Variables

Para desarrollar un modelo de Regresión Logística sólido, es fundamental seleccionar variables que no solo tengan poder predictivo, sino que también respeten los supuestos del modelo (como la ausencia

de multicolinealidad).

Para este caso, hemos elegido Análisis de Correlación e Información Mutua por las siguientes razones:

- Análisis de Correlación (Filtro de Multicolinealidad): La regresión logística falla si las variables predictoras están muy relacionadas entre sí (ej. `payments_initiated` vs `payments_completed`). La correlación nos permite detectar estas redundancias y eliminar variables que inflarían el error del modelo.
- Información Mutua (Captura de Relaciones No Lineales): A diferencia de la correlación estándar, la Información Mutua mide cuánto reduce la incertidumbre de la variable `is_churned` el conocer otra variable. Es ideal aquí porque captura relaciones complejas y funciona tanto para variables numéricas como categóricas, ayudándonos a identificar predictores potentes que un análisis lineal podría pasar por alto.

Descartamos PCA porque mezcla las variables originales, perdiendo la capacidad de interpretar qué acción concreta (ej. "aumentar pagos") previene el churn, que es el objetivo del proyecto.

```
In [35]: df_num <- df_clean %>% select(where(is.numeric))
```

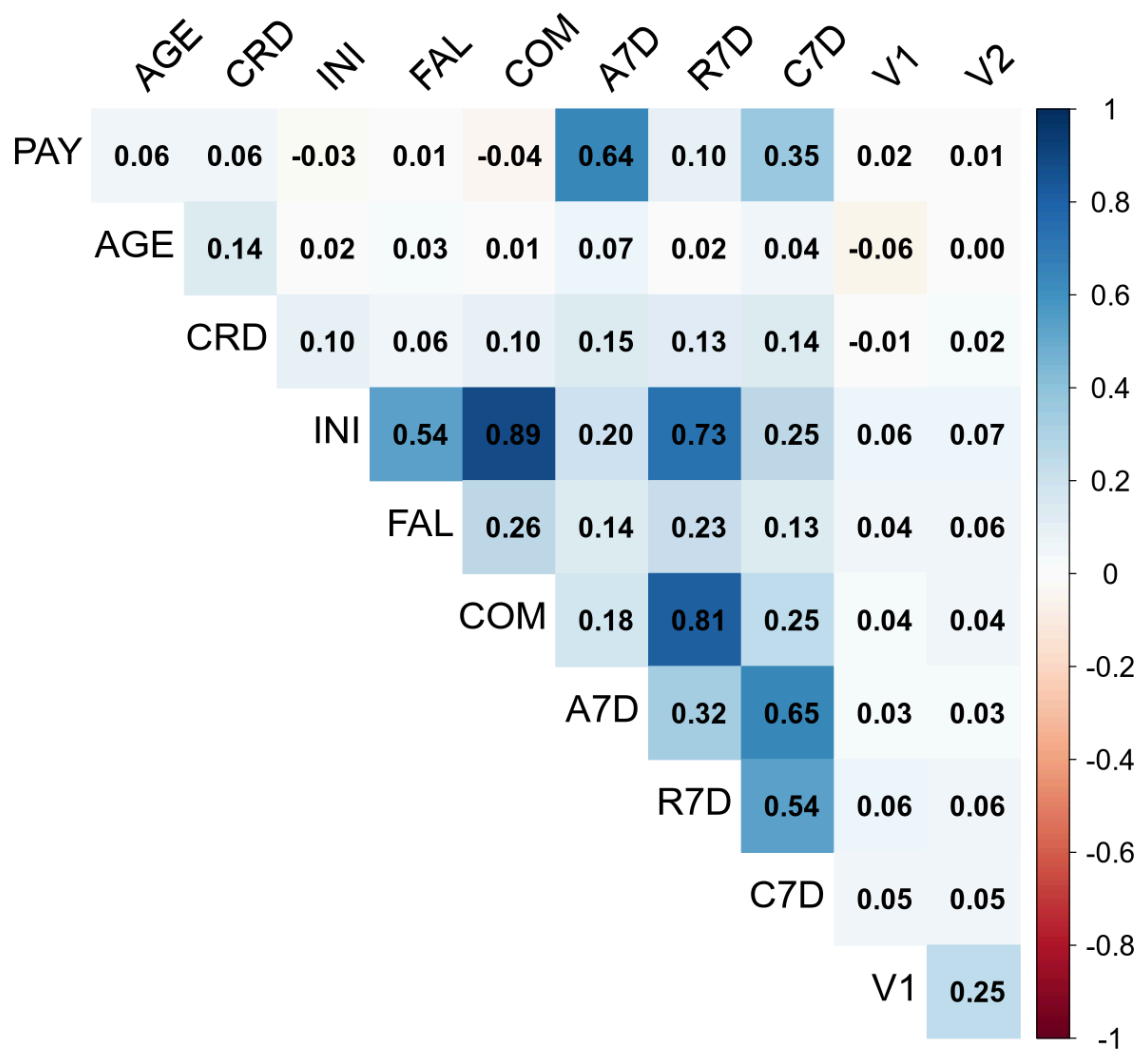
```
In [36]: options(repr.plot.width = 12, repr.plot.height = 12)
```

```
df_mini <- df_num
colnames(df_mini) <- c(
  "PAY", "AGE", "CRD",
  "INI", "FAL", "COM", "A7D", "R7D",
  "C7D", "V1", "V2"
)

res <- cor(df_mini, use = "pairwise.complete.obs")

par(mar = c(0, 0, 0, 0))

corrplot(res,
  method = "color",
  type = "upper",
  addCoef.col = "black",
  tl.col = "black",
  tl.srt = 45,
  tl.cex = 2,
  number.cex = 1.5,
  cl.cex = 1.5,
  diag = FALSE,
  mar = c(0, 0, 0, 0)
)
```



```
In [37]: df_discreto <- discretize(df_num)
```

```
In [38]: ranking_im <- data.frame(
  Variable = colnames(df_num),
  Importancia = sapply(
    colnames(df_num),
    function(x) {
      mutinformation(
        df_discreto[[x]],
        df_clean$is_churned
      )
    }
  )
) %>% arrange(desc(Importancia))
```

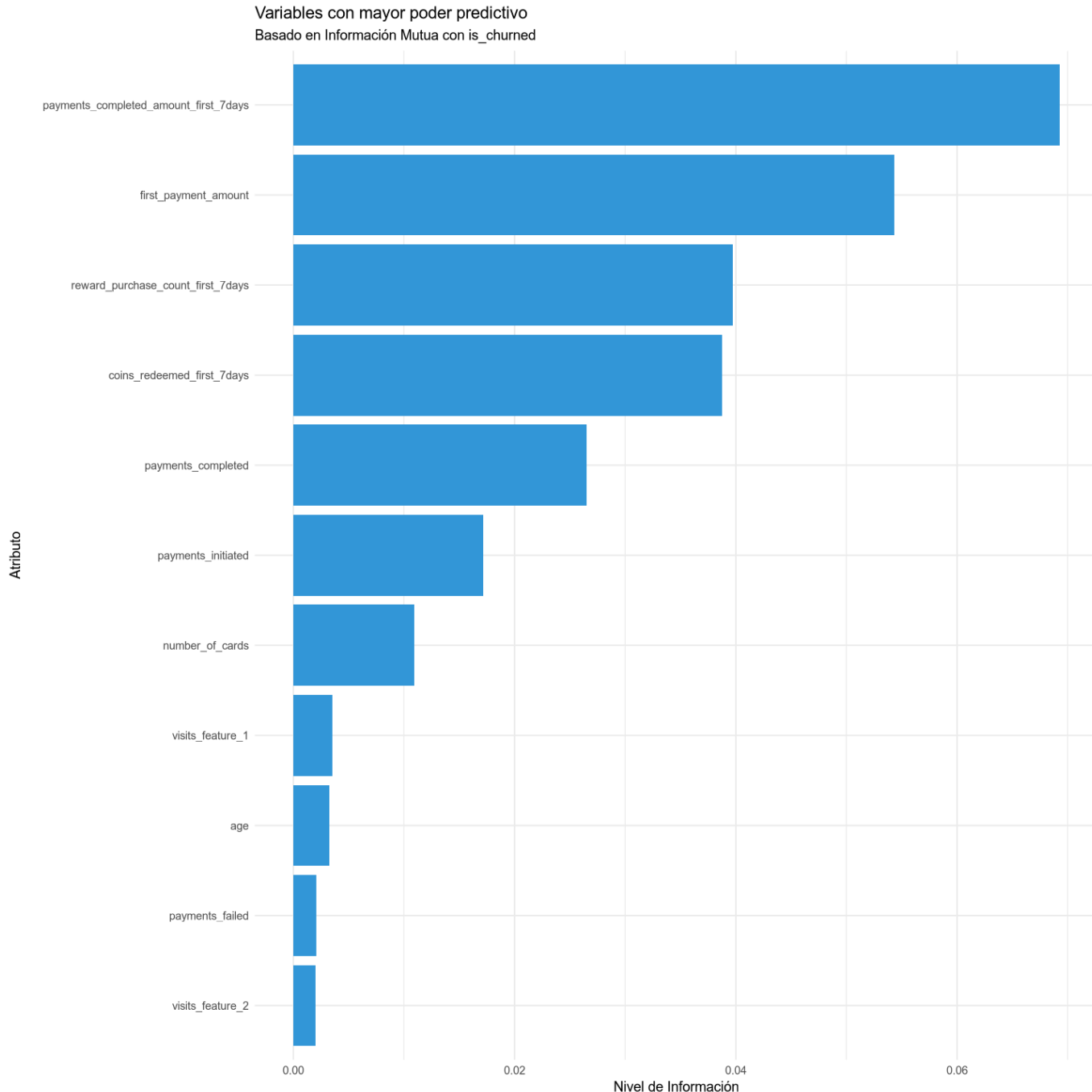
```
In [39]: print("Ranking de importancia (Información Mutua):")
print(ranking_im)
```

```
[1] "Ranking de importancia (Información Mutua):"
```

	Variable
payments_completed_amount_first_7days	payments_completed_amount_first_7days
first_payment_amount	first_payment_amount
reward_purchase_count_first_7days	reward_purchase_count_first_7days
coins_redeemed_first_7days	coins_redeemed_first_7days
payments_completed	payments_completed
payments_initiated	payments_initiated
number_of_cards	number_of_cards
visits_feature_1	visits_feature_1
age	age
payments_failed	payments_failed
visits_feature_2	visits_feature_2

	Importancia
payments_completed_amount_first_7days	0.069267281
first_payment_amount	0.054355181
reward_purchase_count_first_7days	0.039754582
coins_redeemed_first_7days	0.038756468
payments_completed	0.026540842
payments_initiated	0.017160313
number_of_cards	0.010953377
visits_feature_1	0.003529970
age	0.003226934
payments_failed	0.002104456
visits_feature_2	0.002000001

```
In [40]: ggplot(ranking_im, aes(x = reorder(Variable, Importancia), y = Importancia)) +  
  geom_bar(stat = "identity", fill = "#3498db") +  
  coord_flip() +  
  labs(  
    title = "Variables con mayor poder predictivo",  
    subtitle = "Basado en Información Mutua con is_churned",  
    x = "Atributo", y = "Nivel de Información"  
  ) +  
  theme_minimal()
```



Selección de Variables Predictoras

Hemos elegido las siguientes variables como las más potentes para predecir el Churn:

- **payments_completed_amount_first_7days (A7D):** Es la variable "maestra". Tiene el nivel de información mutua más alto (0.073), confirmando que el volumen económico de la primera semana es el mejor predictor de si un cliente se quedará.
- **first_payment_amount (PAY):** Segunda en el ranking de importancia. Aunque correlaciona con el gasto total (0.64), aporta un matiz distinto: el compromiso inicial del usuario en el momento justo de su registro.
- **reward_purchase_count_first_7days (R7D):** Presenta una importancia alta (0.041). Es un gran indicador de "engagement" o uso activo de los beneficios de la aplicación.
- **coins_redeemed_first_7days (C7D):** Al igual que las recompensas, el uso de monedas virtuales demuestra que el usuario ha explorado las funcionalidades de valor de la app.
- **payments_completed (COM):** Representa la frecuencia de uso. Es preferible a otros indicadores de actividad por su mayor peso en el ranking de IM.

Justificación de las variables descartadas (Limpieza de Multicolinealidad)

Para que la Regresión Logística sea fiable y sus coeficientes no se vuelvan locos, hemos decidido eliminar:

- `payments_initiated` (INI): Presenta una correlación crítica de 0.89 con los pagos completados (`COM`). Al ser variables casi idénticas, mantener ambas causaría multicolinealidad. Elegimos `payments_completed` porque su relación con el éxito del cliente es más directa y tiene mayor IM.
- Variable `reward_purchase_count` vs `payments_completed`: Observamos una correlación de 0.81. En versiones finales del modelo, vigilarémos si estas dos variables se "pelean", aunque de momento mantendremos las recompensas por su alto valor estratégico.
- `age`, `visits_feature_2` y `payments_failed`: Se descartan por su bajísima Información Mutua (cerca de 0). Incluirlas solo añadiría "ruido" al modelo sin mejorar su capacidad de acierto.

Con esta selección reducida y potente, nuestro modelo será más sencillo de interpretar, más rápido de ejecutar y, sobre todo, más preciso al evitar datos redundantes.

Desarrollo del modelo de regresión logística

Una vez seleccionadas las variables con mayor poder predictivo y habiendo eliminado las redundancias técnicas, entramos en la fase de Modelado Predictivo. El objetivo es entrenar un algoritmo que aprenda de los datos históricos para asignar a cada cliente una probabilidad de abandono (`churn`).

Selección de la métrica de optimización

Para un problema de detección de Churn, la Exactitud (Accuracy) es una métrica engañosa. Dado que el 71.28% de nuestros clientes no se van, un modelo "perezoso" que dijera que nadie se va acertaría el 71% de las veces, pero sería inútil para el negocio al no detectar ninguna fuga.

Por ello, utilizaremos el AUC-ROC (Área bajo la curva ROC) como métrica principal. Esta evalúa la capacidad del modelo para distinguir correctamente entre un cliente que se va y uno que se queda. También vigilarémos la Matriz de Confusión para entender el Recall (cuántos clientes que realmente se fueron logramos detectar).

Desarrollo del modelo y optimización de categorías

En nuestras pruebas iniciales, detectamos dos problemas críticos al incluir las variables `city` y `device`:

1. El sistema intentaba procesar cientos de ciudades y marcas, saturando la CPU y disparando el tiempo de ejecución.
2. Al dividir los datos, algunas ciudades con muy pocos registros aparecían en el grupo de prueba pero no en el de entrenamiento, haciendo que el modelo fallara al intentar predecir casos que nunca había visto.

Por lo cual hemos agrupado todas las ciudades con menos de 500 clientes y las marcas con menos de 1000 clientes bajo la etiqueta "Other". Esto no solo estabiliza el modelo y evita el overfitting, sino que permite que el entrenamiento sea instantáneo sin perder la información de los grandes núcleos urbanos y las marcas principales.

```
In [41]: df_model <- df_clean %>%
  group_by(city) %>%
  mutate(city = ifelse(n() > 500, as.character(city), "Other")) %>%
  ungroup() %>%
  group_by(device) %>%
  mutate(device = ifelse(n() > 1000, as.character(device), "Other")) %>%
  ungroup() %>%
```

```
mutate(
  city = as.factor(city),
  device = as.factor(device)
)
```

```
In [42]: set.seed(456)
train_index <- createDataPartition(df_model$is_churned, p = 0.8, list = FALSE)
train_set <- df_model[train_index, ]
test_set <- df_model[-train_index, ]
```

```
In [43]: m1_formula <- is_churned ~ payments_completed_amount_first_7days +
  first_payment_amount +
  reward_purchase_count_first_7days +
  coins_redeemed_first_7days +
  payments_completed
model_1 <- glm(m1_formula, data = train_set, family = binomial)
```

Warning message:
"glm.fit: fitted probabilities numerically 0 or 1 occurred"

```
In [44]: m2_formula <- update(m1_formula, . ~ . + city + device + given_permission_1)
model_2 <- glm(m2_formula, data = train_set, family = binomial)
```

Warning message:
"glm.fit: fitted probabilities numerically 0 or 1 occurred"

```
In [45]: prob_m1 <- predict(model_1, newdata = test_set, type = "response")
prob_m2 <- predict(model_2, newdata = test_set, type = "response")
```

```
In [46]: pred_m1 <- factor(ifelse(prob_m1 > 0.5, 1, 0), levels = c("0", "1"))
pred_m2 <- factor(ifelse(prob_m2 > 0.5, 1, 0), levels = c("0", "1"))
```

```
In [47]: print("MATRIZ DE CONFUSIÓN: MODELO 1 (SOLO NUMÉRICO)")
conf_m1 <- confusionMatrix(pred_m1, test_set$is_churned, positive = "1")
print(conf_m1)
```

[1] "MATRIZ DE CONFUSIÓN: MODELO 1 (SOLO NUMÉRICO)"
Confusion Matrix and Statistics

	Reference	
Prediction	0	1
0	14751	5814
1	3	0

Accuracy : 0.7172
95% CI : (0.711, 0.7233)
No Information Rate : 0.7173
P-Value [Acc > NIR] : 0.5221

Kappa : -3e-04

McNemar's Test P-Value : <2e-16

Sensitivity : 0.0000000
Specificity : 0.9997967
Pos Pred Value : 0.0000000
Neg Pred Value : 0.7172867
Prevalence : 0.2826721
Detection Rate : 0.0000000
Detection Prevalence : 0.0001459
Balanced Accuracy : 0.4998983

'Positive' Class : 1

```
In [48]: print("MATRIZ DE CONFUSIÓN: MODELO 2")
conf_out <- confusionMatrix(pred_m2, test_set$is_churned, positive = "1")
print(conf_out)
```

```
[1] "MATRIZ DE CONFUSIÓN: MODELO 2"
Confusion Matrix and Statistics
```

```

      Reference
Prediction    0     1
      0 14259  4782
      1   495  1032

      Accuracy : 0.7434
      95% CI   : (0.7374, 0.7494)
No Information Rate : 0.7173
P-Value [Acc > NIR] : < 2.2e-16

      Kappa   : 0.1854
```

```
McNemar's Test P-Value : < 2.2e-16
```

```

      Sensitivity : 0.17750
      Specificity : 0.96645
      Pos Pred Value : 0.67583
      Neg Pred Value : 0.74886
      Prevalence : 0.28267
      Detection Rate : 0.05018
      Detection Prevalence : 0.07424
      Balanced Accuracy : 0.57198
```

```
'Positive' Class : 1
```

```
In [49]: roc_m1 <- roc(test_set$is_churned, prob_m1, quiet = TRUE)
roc_m2 <- roc(test_set$is_churned, prob_m2, quiet = TRUE)
```

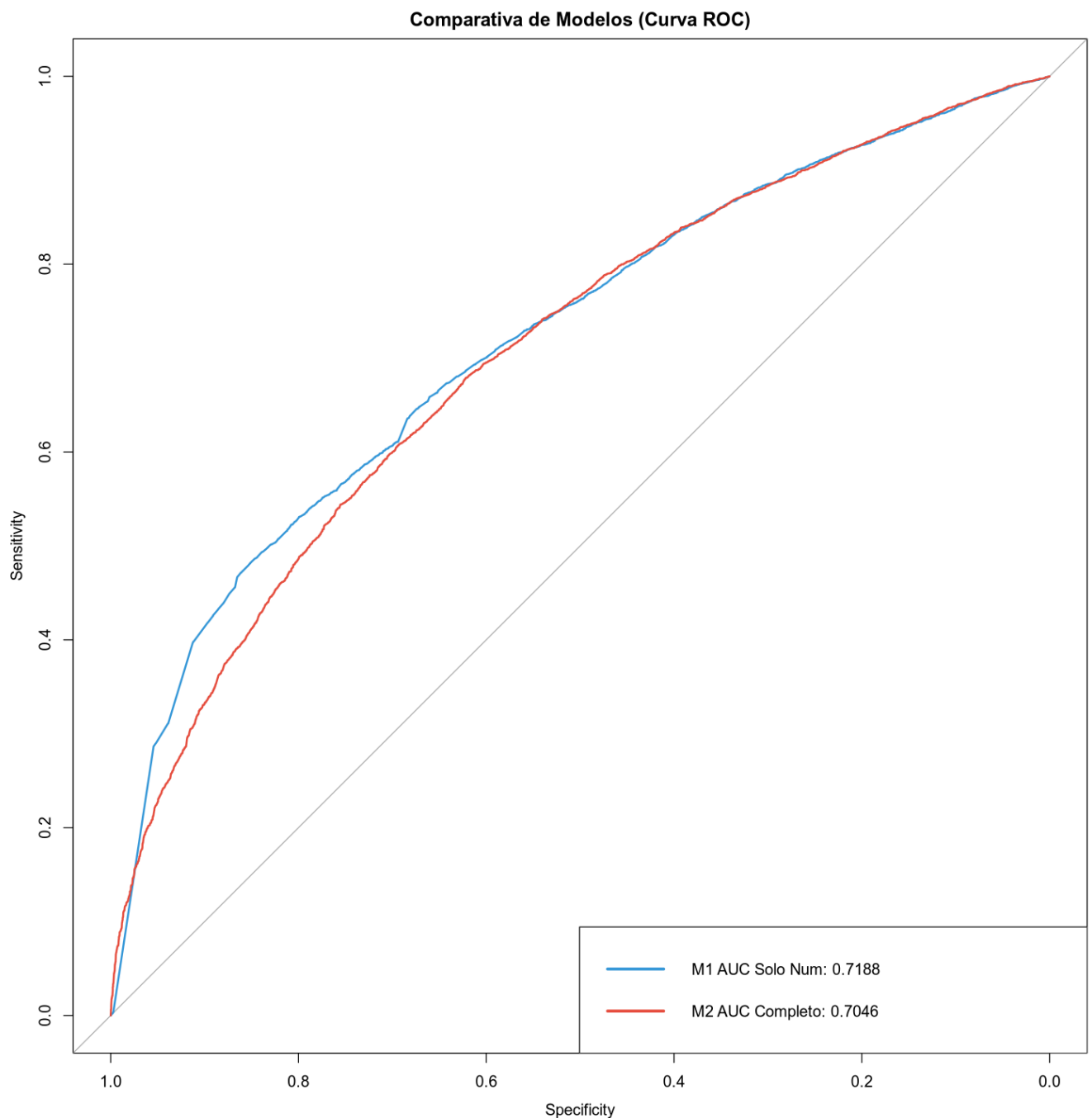
```
In [50]: auc_m1 <- round(as.numeric(auc(roc_m1)), 4)
auc_m2 <- round(as.numeric(auc(roc_m2)), 4)
```

```
In [51]: p_recall <- data.frame(
  Modelo = c("M1: Solo Numérico", "M2: Completo"),
  Recall = c(conf_m1$byClass["Sensitivity"], conf_out$byClass["Sensitivity"]),
  AUC = c(auc_m1, auc_m2)
)
```

```
In [52]: print("Comparativa de métricas clave:")
print(p_recall)
```

```
[1] "Comparativa de métricas clave:"
      Modelo      Recall      AUC
1 M1: Solo Numérico 0.0000000 0.7188
2      M2: Completo 0.1775026 0.7046
```

```
In [53]: plot(roc_m1, col = "#3498db", lwd = 2, main = "Comparativa de Modelos (Curva ROC)")
lines(roc_m2, col = "#e74c3c", lwd = 2)
legend("bottomright",
  legend = c(
    paste("M1 AUC Solo Num:", auc_m1),
    paste("M2 AUC Completo:", auc_m2)
  ),
  col = c("#3498db", "#e74c3c"), lwd = 3
)
```

Análisis comparativo de los modelos:

Tras analizar las métricas de ambos modelos, nos encontramos ante un escenario donde la utilidad práctica define claramente cuál es el modelo ganador:

1. Capacidad Teórica (AUC-ROC)

- Modelo 1 (Solo Numérico): 0.7188
- Modelo 2 (Completo): 0.7046

Si miramos exclusivamente el AUC, el Modelo 1 parece ligeramente superior, indicando una mayor capacidad teórica para ordenar las probabilidades. Sin embargo, en problemas de clases desbalanceadas como el Churn, el AUC puede ser un "espejismo" si no revisamos la matriz de confusión.

2. Detección Real de Fugas (Recall/Sensibilidad) y Exactitud (Accuracy)

Aquí es donde el análisis revela la verdadera utilidad de cada modelo:

- Modelo 1 - Recall nulo (0.00%): A pesar de su AUC aceptable, el Modelo 1 es inútil en la práctica con un umbral de decisión de 0.5. Es tan "conservador" que básicamente asume que nadie va a hacer churn. De los 5.814 clientes reales que abandonaron, no detectó absolutamente a ninguno (solo

predijo que 3 personas se irían, y en las tres falló). Su exactitud (Accuracy) del 71.72% se debe únicamente a que la mayoría de los clientes se quedan, igualando al "No Information Rate".

- Modelo 2 - Detección Activa (17.75%): Aunque su AUC bajó ligeramente, el Modelo 2 sí aprendió a identificar los patrones de fuga. Logra detectar correctamente a 1.032 clientes que van a abandonar la compañía (casi un 18% de los fugados). Además, su Exactitud global (Accuracy) sube al 74.34%, superando por fin la barrera de predecir siempre a la mayoría. Esto demuestra que variables como la ciudad, el dispositivo y los permisos son la clave que permite al modelo "atreverse" a señalar quién está en riesgo.

3. Conclusión y Recomendación Estratégica

- El ganador absoluto para el negocio es el Modelo 2. Es el único de los dos que sirve como herramienta real para lanzar campañas de retención temprana, ya que detecta perfiles de riesgo que el modelo numérico ignora por completo.
- El Recall nulo del Modelo 1 nos indica que el comportamiento financiero o de uso de la app (pagos, monedas) no genera una probabilidad lo bastante alta como para superar el umbral del 50%. El Modelo 2, al añadir el contexto geográfico y técnico del usuario, rompe esa barrera. Aun así, un Recall del ~18% nos indica que para el futuro deberíamos probar modelos más complejos que una regresión logística (como Random Forest o XGBoost) o ajustar el umbral de probabilidad para capturar a más clientes en riesgo.

Recomendaciones Estratégicas para la Reducción de Churn

¿Cómo nos ayuda el modelo a llegar a estas conclusiones?

El modelo predictivo no es solo una herramienta para adivinar quién se va a ir, sino una radiografía del comportamiento de nuestros clientes. El análisis comparativo entre el Modelo 1 (solo variables financieras) y el Modelo 2 (que incluye contexto y comportamiento) nos dio la clave principal: el churn no depende únicamente de cuánto gasta un usuario, sino de su nivel de interacción inicial y su perfil tecnológico/demográfico.

El hecho de que el Modelo 2 lograra detectar clientes en riesgo (pasando de un Recall de 0% a casi un 18%) al incluir variables como city, device y given_permission_1, nos demuestra matemáticamente que la retención está ligada a la fricción tecnológica, la ubicación y la confianza (dar permisos), especialmente concentrado en los primeros 7 días de vida del usuario.

Recomendaciones para la empresa y su justificación

Basándonos en las variables que el modelo ha identificado como determinantes para predecir el abandono, proponemos las siguientes acciones:

1. Foco absoluto en el Onboarding (Los primeros 7 días)

- Gran parte de las variables predictivas más fuertes ocurren en la primera semana (payments_completed_amount_first_7days, reward_purchase_count_first_7days). El modelo nos grita que la decisión de abandonar la app se toma de forma casi inmediata.
- Recomendaría diseñar un flujo de bienvenida mucho más guiado y agresivo. No podemos esperar semanas para enamorar al cliente, hay que asegurar que realice su primera transacción o reclame su primera recompensa en los primeros 3 días.

2. Impulsar la adopción del sistema de recompensas (Coins/Rewards)

- Los datos muestran que la falta de uso del sistema de monedas correlaciona directamente con el abandono. Los usuarios que no interactúan con este sistema tienen mayor probabilidad de ser clasificados como "en riesgo".
- Crearía tutoriales interactivos que fueren (de manera amigable) al nuevo usuario a usar el sistema. Por ejemplo, regalar monedas de bienvenida que caduquen a las 48 horas para generar el hábito y mostrar el valor de la app rápidamente.

3. Mejorar la comunicación al solicitar permisos

- La variable `given_permission_1` resultó ser clave en el Modelo 2 para diferenciar a los usuarios retenidos de los fugados. Un usuario que no da permisos es un usuario desconectado.
- Rediseñaría las pantallas donde se piden permisos (notificaciones, ubicación, etc.). En lugar de lanzar el mensaje por defecto del sistema operativo, explicar primero el beneficio: Activa las notificaciones para recibir alertas de tus ofertas personalizadas.

4. Estrategias de "Perfil Completo" (Reducir los perfiles Unknown)

- Durante la limpieza y exploración de datos, detectamos que los usuarios cuyas variables categóricas (ciudad o dispositivo) eran "Unknown" presentaban una tasa de churn altísima (42.5%). El anonimato es síntoma de bajo compromiso.
- Ofrecería micro-incentivos (como descuentos o monedas extra) a cambio de que el usuario complete su perfil. Tener mejores datos no solo fideliza al cliente por la recompensa, sino que alimentará a nuestro modelo para que sea aún más preciso en el futuro.

5. Auditoría técnica y localizada

- Al ver que `device` y `city` activaron el poder predictivo del Modelo 2, nos indica que el abandono tiene un fuerte componente técnico o geográfico (como vimos con regiones específicas que concentraban el abandono).
- Diría al equipo de IT que debe investigar si la aplicación tiene bugs, lentitud o fallos en los dispositivos específicos asociados al churn. Asimismo, el equipo de marketing debe evaluar si las campañas lanzadas en ciertas ciudades están atrayendo a un público que no encuentra valor en la app.